KU LEUVEN GENT

Verslag: GPU chaos game

Geavanceerde computerarchitectuur

Robbe Hovaert
Lukas Hoste
Master in de industriële ingenieurswetenschappen
Academiejaar 2024-2025

1 INTRO

Binnen dit project hebben wij een willekeurige fractaalgenerator gemaakt op de GPU. We bespreken eerst kort de theorie om erna uitleg te geven over onze implementatie. Vervolgens bespreken we tekortkomingen en mogelijke uitbreidingen. We sluiten af met een evaluatie tegenover een CPU implementatie om te kijken of het uitvoeren op de GPU een groot effect heeft op de rekentijd. De code van dit project is te vinden op Github (`https://github.com/LukasHoste/gcomputerarch_project`).

2.1 Fractalen

Een fractaal is een geometrisch object dat op elke schaal nieuwe details toont en waarvan de fractale dimensie strikt groter is dan de topologische dimensie. Dit wordt ook zelfgelijkendheid genoemd. Deze fractalen kunnen zowel wiskundig als natuurlijk zijn.

2.1.1 Sierpiński driekhoek

In dit project zijn wij gestart vanuit de Sierpiński driehoek. Dit is een fractaal met de algemene vorm van een driehoek, met binnen deze driehoek meer driehoeken van dezelfde vorm. Deze fractaal kan op meerdere manieren opgebouwd worden. De Sierpińskidriehoek wordt klassiek verkregen door herhaaldelijk drie affiene transformaties met factor $1/2$ toe te passen, elk gericht op één hoekpunt van een gelijkzijdige driehoek. Dit wordt ook de formele methode genoemd.

2.1.2 Chaos game

De formele methode is niet efficiënt om te berekenen. Een alternatieve manier van werken is de chaos game. We starten met een wolk van punten. Op elk van deze punten wordt willekeurig één van de transformaties bij de formele methode toegepast, elk met een gelijke kans. Als dit heel veel wordt gedaan convergeert de figuur op termijn naar de figuur van de formele methode. Het grote voordeel hier is dat elk punt individueel behandeld kan worden. Hierdoor is dit algoritme ideaal voor parrallelisatie op de GPU. De chaos game laat algemeen ook meer flexibiliteit toe dan de formele methode.

2.1.3 Affiene transformatie

Een affiene transformatie is een type transformatie waarbij lijnen en evenwijdigheid behouden worden, euclidische afstanden en hoeken hoeven echter niet behouden te worden. Voor het werken met afbeeldingen bestaan er standaard matrices voor deze transformatie zodat rotatie, shear, ... kunnen toegepast worden. Als we de punten in de chaos game voorstellen als een 3x1 matrix kunnen we er affiene transformaties op toepassen. Als we starten met willekeurige affiene transformatie matrices kunnen we andere fractalen bekomen dan de Sierpiński driehoek. De aard en hoeveelheid van transformaties maakt niet uit. De enige beperking is dat de absolute waarde van de uiteindelijke schaalfactor kleiner dan één is.

3.1 Punten

Om met affiene matrices te werken moeten we punten voorstellen als een matrix. Bij het maken van een standaard Sierpiński driehoek definiëren we eerst drie matrices voor de hoekpunten van de driehoek.

Linksonder	Rechtsonder	Bovenkant
$\begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0.5 & 0 & 0.5 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0.5 & 0 & 0.25 \\ 0 & 0.5 & 0.5 \\ 0 & 0 & 1 \end{bmatrix}$

Ook de punten waarop operaties uitgevoerd worden moeten gedefinieerd worden. Voor deze punten wordt gebruik gemaakt van een matrix voor de positie en de kleur. Voor de positie wordt een 3x1 matrix gebruikt en voor de kleur een 4x1 matrix.

De laatste rij is bij ons altijd 0...0 1. Bij normale homogene coordinaten kan aan schaling worden gedaan om bv punten die oneindig ver zijn voor te stellen, maar dit doen wij niet. Wij werken met een homogeen coördinatenstelsel omdat translaties ook als matrix vermenigvuldiginen kunnen voorgesteld worden.

Positiematrix (3×1): $\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ Kleurmatrix (4×1): $\begin{bmatrix} r \\ g \\ b \\ 1 \end{bmatrix}$

3.2 Affiene matrices

Om andere fractalen dan de Sierpiński driehoek te bekomen maken we gebruik van Affiene matrices. Deze affiene matrices gebruiken we om andere drie startpunten dan de hoekpunten van een driehoek te definieren. Hiervoor kunnen we gebruik maken van vier types transformaties:

- Schaal
- Translatie
- Rotatie
- Shear

Om verschillende fractalen te bekomen wordt in deze matrices gebruik gemaakt van random waarden.

Schaalmatrix	**Translatiematrix**	**Rotatiematrix**
$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Shearmatrix

$$\begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Symbolen:

s_x, s_y	Schaalfactoren langs de x- en y-as ($0 < s \leq 1$)
t_x, t_y	Translaties (verschuivingen) langs de x- en y-as
θ	Rotatiehoek in radialen, $0 \leq \theta < 2\pi$
sh_x, sh_y	Shear-factoren (scheefstelling in x- en y-richting)

Om de uiteindelijke startpunten te genereren gebruiken we nog een vermenigvuldiging van verschillende affiene matrices (met random waarden). Omdat een vermenigvuldiging van al deze types matrices geen mooie fractalen opleverde werd shear weggelaten.

$$\textbf{Affine matrix (random)} = \text{Scale} \cdot \text{Rotation} \cdot \text{Translation} \tag{3.1}$$

Om de kleuren willekeurig te maken gebruiken we ook affiene matrices. Hiervoor worden de volgende 4x4 matrices gebruikt.

Schaal: $\begin{bmatrix} 2r_1 - 1 & 0 & 0 & 0 \\ 0 & 2r_2 - 1 & 0 & 0 \\ 0 & 0 & 2r_3 - 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ Translatie: $\begin{bmatrix} 1 & 0 & 0 & 2r_4 - 1 \\ 0 & 1 & 0 & 2r_5 - 1 \\ 0 & 0 & 1 & 2r_6 - 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Yaw (Z): $\begin{bmatrix} \cos\theta_y & -\sin\theta_y & 0 & 0 \\ \sin\theta_y & \cos\theta_y & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ Pitch (Y): $\begin{bmatrix} \cos\theta_p & 0 & \sin\theta_p & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta_p & 0 & \cos\theta_p & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ Roll (X): $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta_r & -\sin\theta_r & 0 \\ 0 & \sin\theta_r & \cos\theta_r & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

De Yaw, Pitch en Roll matrices worden gebruikt om een rotatie matrix op te bouwen door ze te vermenigvuldigen. De finale affiene matrix wordt aan de hand van dezelfde formule als deze voor de coördinaten in Vergelijking (3.1). We gebruiken RGB als kleurruimte.

3.3 Werkwijze

In onze implementatie maken wij willekeurige fractalen. Hiervoor vullen we de variabelen in de transformatie matrixes willekeurg in. Er is een uniforme random distributie gebruikt. Voor de hoeken is het domein $[-\pi, +\pi]$. Voor andere variabelen is dit $[-1, 1]$.

Het genereren van een fractaal start door een aantal willekeurige punten te genereren (bijvoorbeeld 4000000). Deze punten worden op willekeurige coördinaten geplaatst en met een willekeurige kleur. Hiervoor maken we gebruik van een array van gekleurde punten (coördinaten en RGB).

Vervolgens beginnen we met het maken van de fractaal aan de hand van een aantal iteraties, voor een afbeelding met een bepaalde hoogte en breedte. Hier worden ten eerste de punten overgekopieerd naar de GPU. Hiervoor maken we gebruik van twee buffers, namelijk een input en een output buffer. Deze worden telkens omgewisseld zodat input data niet overschreven wordt door output. Hierna worden twee buffers aangemaakt voor het opslaan van alle RGB waarden. Een initiele buffer met de waarden voorgesteld als float en een finale buffer voor de effectieve RGB waarden. Vervolgens wordt een buffer gealloceerd voor schalingsdata. Ten laatste maken we een buffer aan om alle resultaten in op te slaan.

Nu kunnen we beginnen met het uitvoeren van iteraties. Hiervoor wordt ten eerste geselecteerd welke buffer als input en welke als output gebruikt moet worden. Hierna wordt de effectieve kernel uitgevoerd.

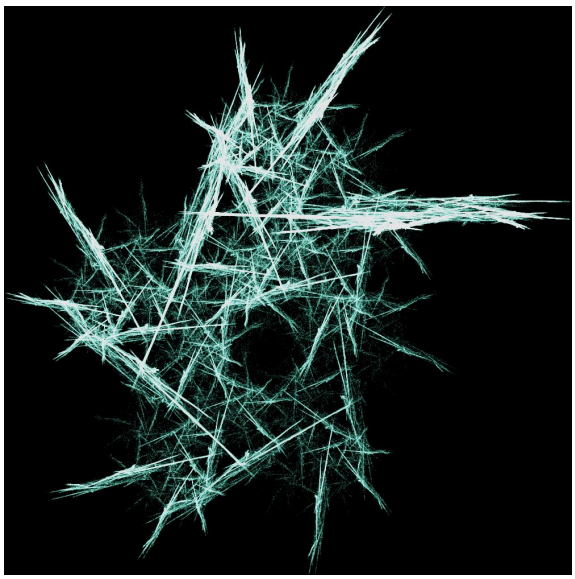
In de kernel worden ten eerste de verschillende affiene matrices opgehaald, deze worden in de main opgeslagen via constant memory omdat deze gedurende de iteraties nooit veranderen. Vervolgens worden in de kernel iteraties uitgevoerd waarbij telkens een random kleurpunt (coördinaat + kleur) geselecteerd wordt. Op deze punten wordt vervolgens een matrix multiplicatie uitgevoerd met één van de gemaakte affiene matrices.

Vervolgens wordt aan de hand van de resulterende punten van de kernel een omzetting gedaan naar float punten, hierin wordt normalisatie en bilinear splatting uitgevoerd. Met bilinear splatting bepaalt een punt de waarde van vier pixels. Hier wordt een schaling van de kleuren op uitgevoerd. Na deze schaling worden de float waarden omgezet naar effectieve RGB waarden. Hiervoor wordt gebruik gemaakt van een "tone mapping" functie. Hierna kunnen deze waarden geconverteerd worden naar een afbeelding van de iteratie, zoals weergegeven in Figuur 3.1.

De tone mapping functie is hieronder gegeven. X is een R, G of B waarde. Y is de output R, G of B waarde. De functie heeft een horizontale asymptoot op de waarde 0.9, waardoor elke mogelijke x waarde een geldige RGB waarde is.

$$y = \left(1 - e^{-0.3 \cdot \mathrm{x}}\right) \cdot 0.9$$

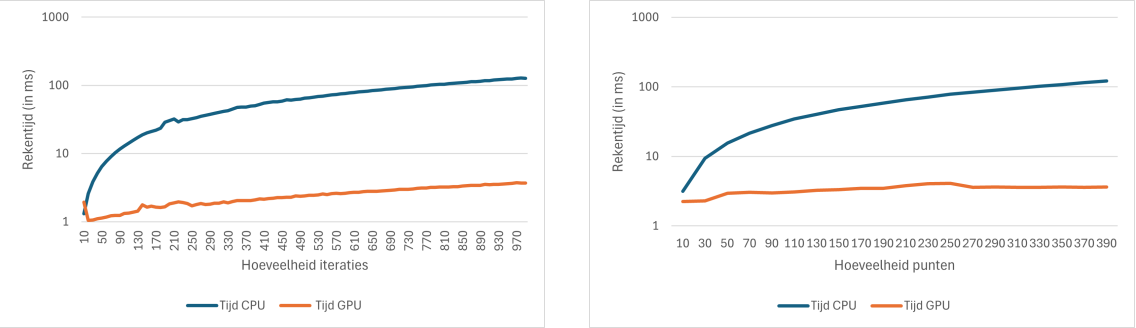
Tot slot maken we in de kernels gebruik van striding. Dit voor wanneer er niet genoeg threads gebruikt worden om alle datapunten in één keer af te handelen.



Figuur 3.1: Voorbeeld van frame 199 van een willekeurige fractaal aan de hand van de chaos game.

Problematisch met de huidige methode is dat bij zeer veel punten de VRAM van de GPU volledig gevuld wordt. Als we een zeer hoge hoeveelheid punten willen gebruiken moet dus de hantering van de punten aangepast worden. Daarnaast worden momenteel minimum en maximum berekeningen op een atomaire manier gedaan. Deze worden gebruikt om de positie en kleuren van punten terug te mappen naar een geldig domein. Wij mappen kleuren en posities met de minimum waarde op nul en de maximum waarde op één. Hiervoor moeten wij de minimum en maximum waarde bepalen van x, y, R, G en B. Hier bestaan zeer efficiënte algoritmen voor op de GPU. Wij gebruiken deze niet en hebben een lockende atomaire implementatie gebruikt.

Om de prestaties te meten hebben we testen uitgevoerd op een CPU en GPU implementatie van het algoritme. De CPU implementatie berekent de locatie en kleur van de punten. De GPU implmentatie doet hierbij nog rasterisatie, schaalcorrectie (zorgen dat alle data zich bevind in een geldig domein) en splatting.



(a) Tegenover iteraties

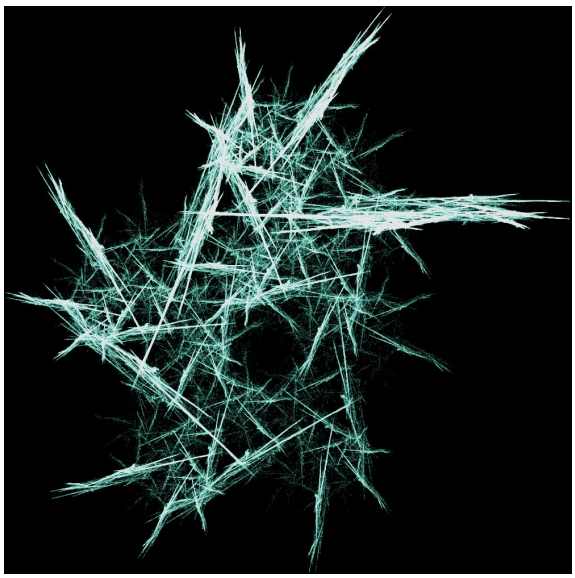
(b) Tegenover punten

Figuur 5.1: Rekentijden tegenover eigenschappen voor de CPU als de GPU implementatie

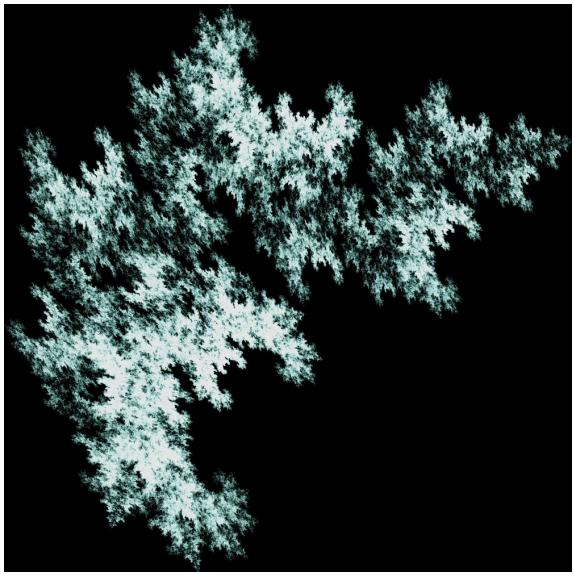
In Figuur 5.1 zien we dat de GPU variant meerdere factoren sneller is dan de CPU variant. Beiden stijgen wel exponentieel. Dit is te verklaren omdat de GPU een locking algoritme gebruikt voor minima en maximas te berekenen. Dit heeft echter weinig invloed. Onze grafiek is logaritmisch, dus het verschil op de rekentijden wijst erop dat de CPU variant exponentieel trager is dan de GPU variant. De hoeveelheid iteraties heeft meer invloed dan de hoeveelheid punten.

6 CONCLUSIE

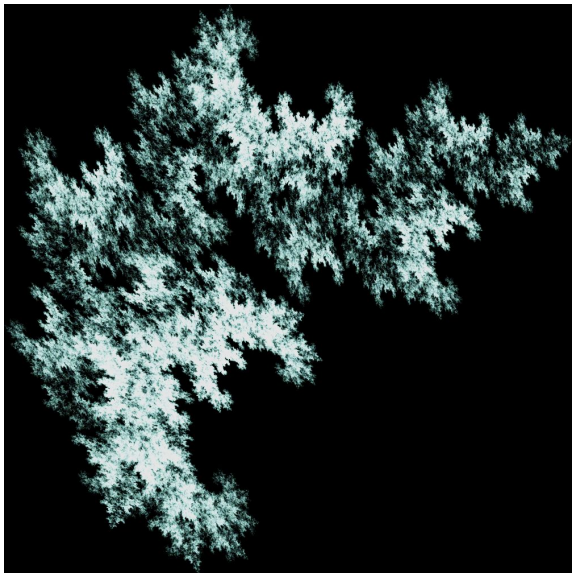
We hebben een goed werkende implementatie van de chaos game voor de GPU ontwikkeld. Om de implementatie zo snel mogelijk te maken, maken we gebruik van constant memory en asynchrone operaties. Uit metingen zien we dat de benodigde tijd op de GPU nog steeds exponentieel stijgt, maar zwakker dan op de CPU. Het is dus zeker nuttig om de chaos game op de GPU uit te voeren wanneer met een hoge resolutie gewerkt wordt en dus meer iteraties en of punten nodig zijn. Wijzigingen van de manier waarop punten opgeslagen worden en het gebruik van locking functies kunnen nog toegevoegd worden.



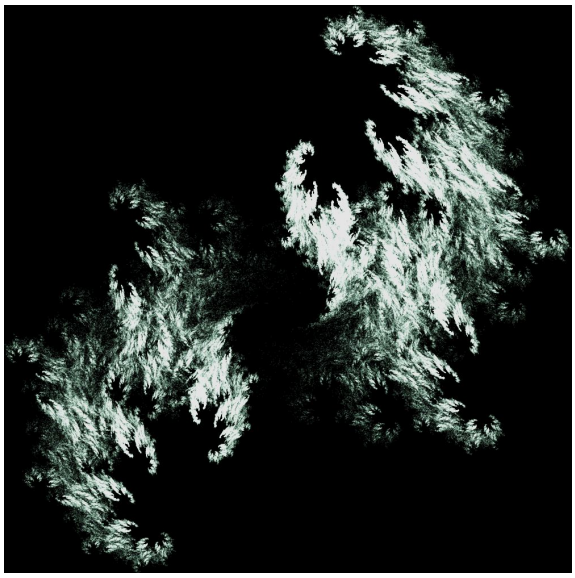
Figuur A.1: Seed: 929290504577



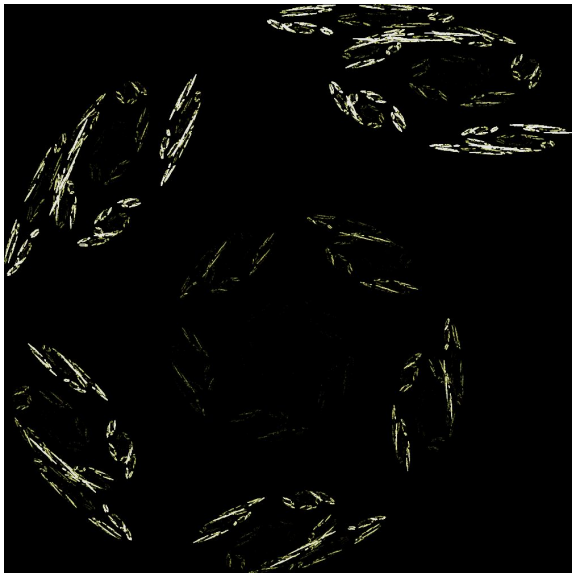
Figuur A.2: Seed: 2468498443987



Figuur A.3: Seed: 2468498443987



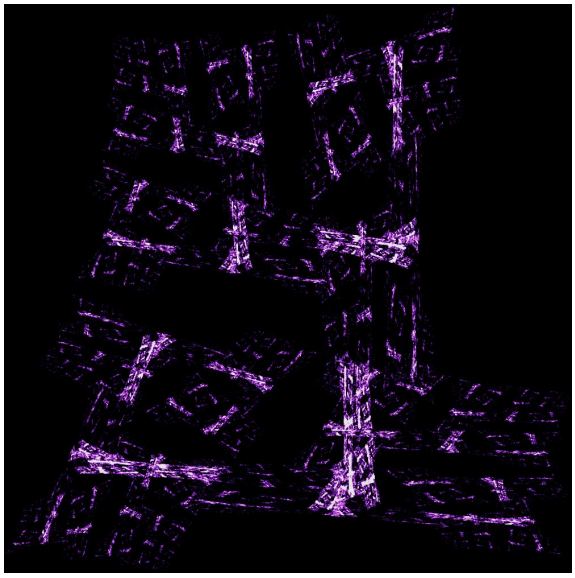
Figuur A.4: Seed: 684114833828686



Figuur A.5: Seed: 242611625460



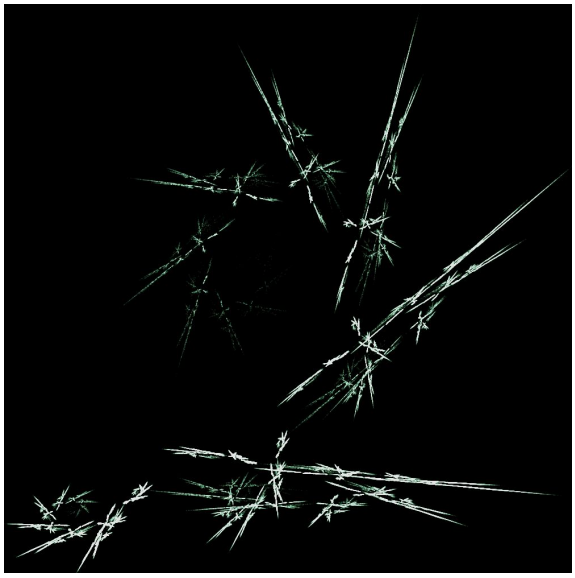
Figuur A.6: Seed: 814348701777



Figuur A.7: Seed: 882057400714



Figuur A.8: Seed: 518492616356



Figuur A.9: Seed: 337771246872